© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_12

12. Quarkus + LangChain4j: Lightning-Fast Gen AI

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to explore and build generative AI (Gen AI) applications using the Quarkus Java framework through LangChain4j integration.

Introduction

In this chapter, you'll harness the power of large language models (LLMs) within the battle-tested, cloud-native framework of Quarkus. In this chapter, the goal is to guide you through the end-to-end process of building generative AI services in Java by leveraging LangChain4j's tight integration with Quarkus. You'll see how to register AI services, craft precise prompts, and wire in external tools—all with minimal boilerplate and maximum performance.

You will begin by seeing how to integrate LangChain4j into a Quarkus project and seeing how simple annotations and dependency injection can transform your REST endpoints into intelligent AI-powered handlers. Next, you'll dive into prompt engineering: learning how system and user messages—defined right in your Java interfaces—give you fine-grained control over model behavior and structured JSON outputs. After that, you'll see how to enrich your AI workflows with custom tools, such as a Panache-driven customer repository, so that your LLM can fetch live data at inference time.

Building on these foundations, you'll explore chat memory in Quarkus, enabling stateful conversations that persist context across calls. Then, you'll introduce the retrieval-augmented generation (RAG) pattern: ingesting your product catalog into a Redis-backed embedding store and combining semantic search with LLM prompting ground responses in real-world data. Throughout, you'll keep performance and scalability top of mind, showing how Quarkus's build-time bootstrapping and lightweight runtime make it the ideal host for high-throughput AI microservices.

By the end of this chapter, you'll have a complete, production-ready blueprint for crafting Java applications that seamlessly blend Quarkus's speed with the creativity of LLMs—empowering you to deliver intelligent, responsive services in the Java ecosystem.

Integration of LangChain4j in Quarkus

LLMs are reshaping software development by transforming how you interact with users and encode business logic. While LLMs have existed for years, their accessibility and power skyrocketed once OpenAI opened up the ChatGPT APIs. Today you'll find hundreds of models on Hugging Face supported by major tech players each offering their own flavors.

The Quarkus team in collaboration with Dmytro Liubarskyi, and the LangChain4j maintainers built a Quarkus extension that brings LLM capabilities natively into your JVM applications in Quarkus. Under the hood, it leverages LangChain4j, a pure-Java re-implementation of the popular LangChain framework, to provide:

- Declarative prompts via @SystemMessage and @UserMessage annotations
- Zero-boilerplate LLM clients for OpenAI, Hugging Face, Ollama, Jlama, etc.
- Tool invocation—annotate any CDI bean method with @Tool for model-driven lookups
- Chat memory and RAG support via Redis or in-memory stores

With this extension, you can instantly power your Quarkus REST endpoints with AI. For example, an application built on Quarkus LangChain4j can:

- Automatically categorize or classify documents
- Extract structured and unstructured data from diverse sources
- Drive chatbots that maintain conversational context
- Generate personalized text—emails, reports, summaries—on demand

To start on your journey on building a generative AI application using Quarkus, let's visit https://code.quarkus.io/ and select the quarkus-langchain4j-openai package, as shown in Figure 12-1.

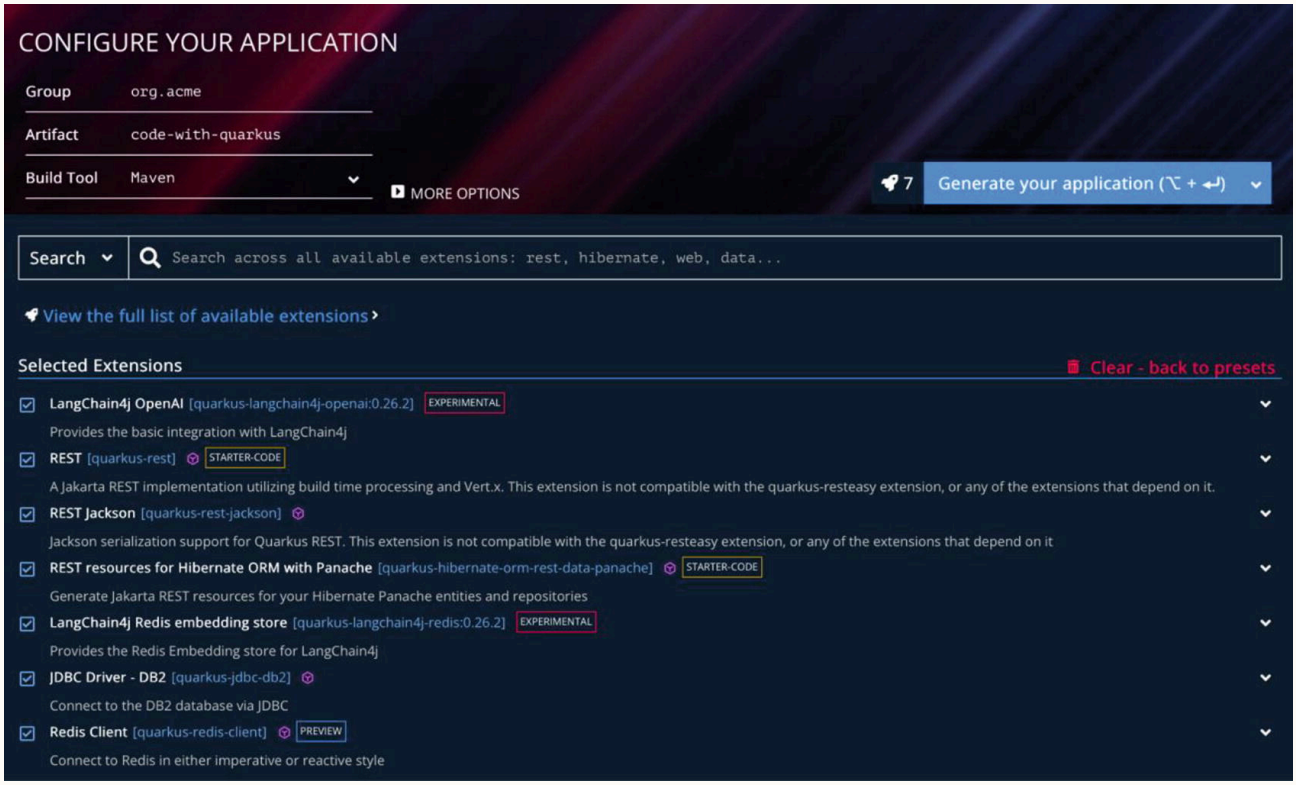


Figure 12-1 Quarkus-langchain4j-openai package in Quarkus Dependency UI

In addition, let's add a few more dependencies needed for the application:

- **Quarkus-rest:** For restful endpoints.
- **Quarkus-langchain4j-openai:** To integrate langchain4j with OpenAI
- **Quarkus-rest-jackson:** For JSON encoding purposes
- **Quarkus-hibernate-orm-rest-data-panache:** ORM layer for querying relational database
- **Quarkus-langchain4j-redis:** Using Redis as a vector store
- **Quarkus-jdbc-db2:** db2 as a relational database
- **Quarkus-redis-client:** Redis client connection

Please do not get overwhelmed with the package list. You will start slowly and learn the ropes of Gen AI in Quarkus. Download the zip archive and decompress it and load it in your favorite IDE of choice. If you are using IntelliJ IDE, it comes preloaded with the Quarkus plugin. In VS Code, you can download the official Quarkus extension from Red Hat, which is helpful to start building and running Quarkus applications. The following is the list of dependencies in your pom.xml file:

File: pom.xml

```
<dependency>
   <groupId>io.quarkus</groupId>
   <artifactId>quarkus-rest</artifactId>
</dependency>
```

```
<dependency>
  <groupId>io.quarkiverse.langchain4j</groupId>
  <artifactId>quarkus-langchain4j-openai</artifactId>
  <version>0.26.2</version>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-rest-jackson</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-hibernate-orm-rest-data-panache</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkiverse.langchain4j</groupId>
  <artifactId>quarkus-langchain4j-redis</artifactId>
  <version>0.26.2</version>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-jdbc-db2</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-redis-client</artifactId>
</dependency>
```

The next step is to add a few configuration properties to your application.properties file to make it ready for integrating LangChain4j and OpenAI with Quarkus.

Log requests and log responses allow you to log any requests happening through LangChain4j in the terminal, interface which is helpful for debugging while you are getting started. The openai.chat-model.temperature package sets the temperature for all the OpenAI requests. The Openai.api-key package takes an OpenAI API key, and redis.dimension takes in the value of the embeddings dimension for vectors it will be storing, which is 1536 for OpenAI-based embedding model.

File: src/main/resources/application.properties

```
quarkus.langchain4j.openai.chat-model.temperature=0.3
quarkus.langchain4j.timeout=120s
quarkus.langchain4j.log-requests=true
quarkus.langchain4j.log-responses=true
quarkus.langchain4j.openai.api-key=YOUR-OPENAI-API-KEY
```

```
quarkus.langchain4j.redis.dimension=1536
```

Once these steps are done, Quarkus will bootstrap your LLM client, wire in any tools or memory providers you declare, and expose your interface methods as AI-powered handlers—letting you focus on prompts and business logic instead of connection plumbing.

Prompt Engineering in Quarkus

In this section, you'll see how Quarkus and LangChain4j let you externalize your LLM prompts directly in Java interfaces and annotations—turning your business logic into clean, declarative AI workflows. Say you work at an ecommerce company and want to categorize customer comment feedback about your products. Figure 12-2 illustrates this flow.

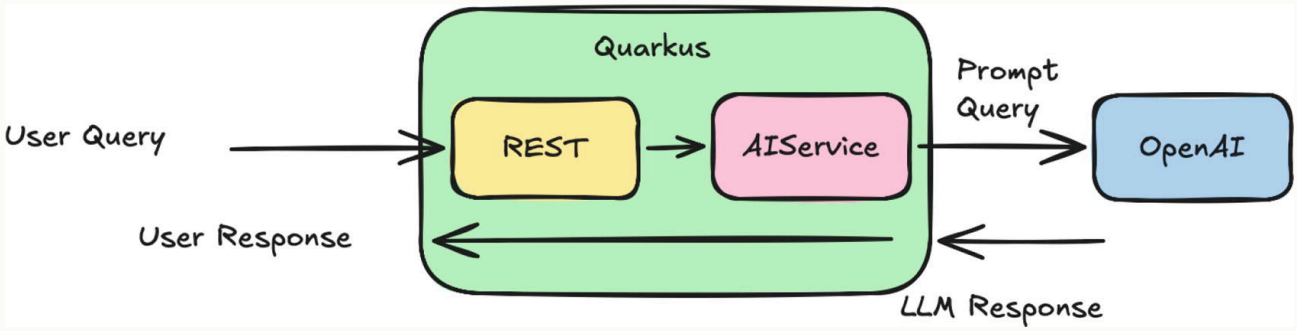


Figure 12-2 Prompt engineering flow in Quarkus with LangChain4j extension

After installing the extension, the next step is to define your desired LLM behavior. The Quarkus LangChain4j extension helps you describe these interactions declaratively—much like the Quarkus REST client does for HTTP calls. You represent the conversation through an interface annotated with @RegisterAiService:

```
@RegisterAiService
public interface SupportService {
   // methods
}
```

Your application would then be able to use the LLM by injecting the SupportService interface and calling the respective methods defined.

Crafting Prompts with Annotations

Referring to methods, they are the core engine that drives all the functionality. You will describe what you want the LLM to do using natural language. First, you start with

@SystemMessage to define the role and scope. Then, you can use @UserMessage to describe the functionality at hand.

File: src/main/java/io/demo/langchain4j/sample/service/SupportService.java

```
package io.demo.langchain4j.sample.service;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import io.quarkiverse.langchain4j.RegisterAiService;
// Marks this interface as an AI service: Quarkus will create an
// implementation that orchestrates calls to your chosen LLM provider.
@RegisterAiService
public interface SupportService {
   // Sets the overall "persona" and task context for the LLM (its
  // system role). It's sent once at the start of each session.
  @SystemMessage("""
      You are working for an ecommerce company, processing
      comments about their products and providing additional
      information as requested by the customer. Categorize
      comments into positive, neutral and negative ones,
      responding with a JSON format document.
""")
   // Defines the actual prompt template. You can:
  // Embed examples (few-shot prompting) to show the model the
  // expected format.
  // Use placeholders ({comment}, {customerId}) to inject runtime
  // data.
  // Specify output constraints (here: "Respond with a JSON
  // document...").
  @UserMessage("""
      Your task is to process the comment delimited by ---.
      Apply sentiment analysis to the comment to determine
      if it is positive, neutral or negative.
      For example:
      - `I love the user experience in your app, you are my
        best place to be!` is a 'POSITIVE'
        comment
      - `J'adore ton application` is a 'POSITIVE' comment
      - `I don't want to use your app, it's too slow!` is a
       'NEGATIVE' comment
      Respond with a JSON format document containing:
      - the 'rating' key set to 'POSITIVE' if the comment
        is positive, 'NEUTRAL' if the comment is neutral,
        'NEGATIVE' otherwise
      - the 'message' key set to a message thanking or
```

```
        apologizing to the customer.
        ---
        {comment}
        ---
    """)
    // The single String comment parameter is bound to {comment} in
    // the template. (Later, we can overload or extend this to accept
    // a long customerId for tool integration.)
    SupportComment categorize(String comment);
}
```

And that's it—you've completed the LLM interaction setup. These directives are based on a set of guiding principles to steer the model's responses.

Exposing an AI-Powered REST Endpoint

To invoke the LLM in your application code, simply inject the SupportService and invoke its categorize method:

File: src/main/java/io/demo/langchain4j/sample/controller/CommentResource.java

```
package io.demo.langchain4j.sample.controller;
import io.demo.langchain4j.sample.service.Comment;
import io.demo.langchain4j.sample.service.SupportService;
import io.demo.langchain4j.sample.service.CommentReview;
import jakarta.inject.Inject;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
// Defines a standard JAX-RS endpoint that accepts a JSON body mapped
// to our Comment record.
@Path("/comment")
public class CommentResource {
    // Quarkus will automatically provide an implementation of the
    // SupportService interface—backed by LangChain4J—so you don't have
    // to manually wire any HTTP clients or JSON parsers.
    @Inject
    SupportService categorize;
    @POST
    public CommentReview categorize(Comment comment) {
        return categorize.categorize(comment.comment());
    }
}
```

Domain Types for Input and Output

File: src/main/java/io/demo/langchain4j/sample/service/Rating.java

```
package io.demo.langchain4j.sample.service;
public enum Rating {
   POSITIVE,
   NEUTRAL,
   NEGATIVE;
}
```

File: src/main/java/io/demo/langchain4j/sample/service/Comment.java

```
package io.demo.langchain4j.sample.service;
import io.quarkus.runtime.annotations.RegisterForReflection;
// Define the Comment record as a top-level public record
// A simple record that carries the raw comment text.
@RegisterForReflection
public record Comment(String comment) {}
```

File: src/main/java/io/demo/langchain4j/sample/service/CommentReview.java

```
package io.demo.langchain4j.sample.service;
import com.fasterxml.jackson.annotation.JsonCreator;
// Holds the model's structured response:
// rating: a POSITIVE, NEUTRAL or NEGATIVE enum
// message: a polite reply to the customer
public record CommentReview(Rating rating, String message) {
  // Ensures Jackson can deserialize the JSON the LLM returns into
  // our record.
  @JsonCreator
   public CommentReview {}
}
```

This is all that needs to be set up to get you up and running. You will now boot up your application and see it in action by invoking a curl-based POST request.

```
./mvnw compile quarkus:dev
```

Output:

```
2025-04-23 05:13:00,589 INFO  [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus
Main Thread) Application starting...
2025-04-23 05:13:00,754 INFO  [io.quarkus] (Quarkus Main Thread) code-with-
quarkus 1.0.0-SNAPSHOT on JVM (powered by Quarkus 3.21.3) started in 149.566s.
Listening on: http://localhost:8080
2025-04-23 05:13:00,756 INFO  [io.quarkus] (Quarkus Main Thread) Profile dev
activated. Live Coding activated.
2025-04-23 05:13:00,757 INFO  [io.quarkus] (Quarkus Main Thread) Installed
features: [agroal, cdi, hibernate-orm, hibernate-orm-panache, hibernate-orm-rest-
data-panache, jdbc-db2, langchain4j, langchain4j-openai, langchain4j-redis, narayana-
jta, qute, redis-client, rest, rest-client, rest-client-jackson, rest-jackson, rest-links,
smallrye-context-propagation, vertx]
```

Quarkus manages creating redis, db2, and other dependent resources for you by using Docker containers. This can be verified by running the following command:

```
docker ps
```

Output:

```
CONTAINER
ID   IMAGE                          COMMAND             CREATED       STATUS       PORTS
                                                NAMES
e95db0bac3d2   icr.io/db2_community/db2:12.1.0.0   "/var/db2_setup/lib/..."   5
minutes ago   Up 5 minutes   22/tcp, 55000/tcp, 60006-60007/tcp, 0.0.0.0:32809-
>50000/tcp, :::32809->50000/tcp   hardcore_dewdney
ad2131dd07f1   redis/redis-stack:latest          "/entrypoint.sh"        5 minutes ago   Up
5 minutes   8001/tcp, 0.0.0.0:32808->6379/tcp, :::32808-
>6379/tcp                      fervent_mirzakhani
be4fefc05e1f   testcontainers/ryuk:0.11.0        "/bin/ryuk"             5 minutes ago   Up
5 minutes   0.0.0.0:32807->8080/tcp, :::32807-
>8080/tcp                      testcontainers-ryuk-25bb0431-c6a1-48e3-ada7-
3ae0b809a24d
```

Let's invoke your POST rest endpoint with a curl request.

```
curl localhost:8080/comment --header 'Content
-Type: text/plain' --data 'This is a great product! I highly recommend it.'
```

Output:

```
{"rating":"POSITIVE","message":"Thank you for your kind words! We're thrilled to hear you love our product."}
```

The application has been able to rightly determine the rating of your feedback. Also, since you have enabled the debugging logs for LangChain4j, you can see the following output in your system logs:

```
2025-04-23 05:19:31,539
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-thread-1) Request:
- method: POST
- url: https://api.openai.com/v1/chat/completions
- headers: [Accept: application/json], [Authorization: Be...kA], [Content-Type: application/json], [User-Agent: langchain4j-openai], [content-length: 1487]
- body: {
  "model" : "gpt-4o-mini",
  "messages" : [ {
    "role" : "system",
    "content" : "      You are working for an ecommerce company, processing comments about\n      their products and providing additional information as requested\n      \tby the customer. Categorize comments into positive, neutral and negative ones,\nresponding with a JSON format document.\n"
  }, {
    "role" : "user",
    "content" : "      Your task is to process the comment delimited by ---.\n      Apply sentiment analysis to the comment to determine\n      if it is positive, neutral or negative.\n\n      For example:\n      - `I love the user experience in your app, you are my best place to be!` is a 'POSITIVE' comment\n      - `J'adore ton application` is a 'POSITIVE' comment\n      - `I don't want to use your app, it's too slow!` is a 'NEGATIVE' comment\n\n      Respond with a JSON format containing:\n      - the 'rating' key set to 'POSITIVE' if the comment is\n      positive, 'NEUTRAL' if the comment is neutral, 'NEGATIVE' otherwise\n      - the 'message' key set to a message thanking or apologizing\n      to the customer. These messages must be polite.\n\n      ---\n      This is a great product! I highly recommend it.\n      ---\n\nYou must answer strictly in the following JSON format: {\n\"rating\": (type: enum, must be one of [POSITIVE, NEUTRAL, NEGATIVE]),\n\"message\": (type: string)\n}"
  } ],
  "temperature" : 0.5,
  "top_p" : 1.0,
  "presence_penalty" : 0.0,
  "frequency_penalty" : 0.0
}
2025-04-23 05:19:32,521
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-
```

thread-1) Response:
- status code: 200
- headers: [Date: Wed, 23 Apr 2025 05:19:32 GMT], [Content-Type: application/json], [Transfer-Encoding: chunked], [Connection: keep-alive], [access-control-expose-headers: X-Request-ID], [openai-organization: user-eldk0kvytvetb2vdgo2y3nvr], [openai-processing-ms: 502], [openai-version: 2020-10-01], [x-ratelimit-limit-requests: 10000], [x-ratelimit-limit-tokens: 200000], [x-ratelimit-remaining-requests: 9999], [x-ratelimit-remaining-tokens: 199692], [x-ratelimit-reset-requests: 8.64s], [x-ratelimit-reset-tokens: 92ms], [x-request-id: req_da5438d76e2db842f75a62abd6e50cf3], [strict-transport-security: max-age=31536000; includeSubDomains; preload], [cf-cache-status: DYNAMIC], [Set-Cookie: __...ne], [X-Content-Type-Options: nosniff], [Set-Cookie: _c...ne], [Server: cloudflare], [CF-RAY: 934ae80e289c2be9-FRA], [alt-svc: h3=":443"; ma=86400]
- body: {

```
 "id": "chatcmpl-BPMo8L36EnE9muR6OUXOvI48GV19u",
 "object": "chat.completion",
 "created": 1745385572,
 "model": "gpt-4o-mini-2024-07-18",
 "choices": [
  {
   "index": 0,
   "message": {
    "role": "assistant",
    "content": "{\n  \"rating\": \"POSITIVE\",\n  \"message\": \"Thank you for your kind words! We're thrilled to hear you love our product.\"\n}",
    "refusal": null,
    "annotations": []
   },
   "logprobs": null,
   "finish_reason": "stop"
  }
 ],
 "usage": {
  "prompt_tokens": 281,
  "completion_tokens": 32,
  "total_tokens": 313,
  "prompt_tokens_details": {
   "cached_tokens": 0,
   "audio_tokens": 0
  },
  "completion_tokens_details": {
   "reasoning_tokens": 0,
   "audio_tokens": 0,
   "accepted_prediction_tokens": 0,
   "rejected_prediction_tokens": 0
  }
```

```
  },
  "service_tier": "default",
  "system_fingerprint": "fp_dbaca60df0"
}
```

Here you can see the prompt context passed in to the API request to OpenAI and the response from OpenAI. This is handy to see what and how the Quarkus/LangChain4j application is interacting with OpenAI APIs.

So, what happens at runtime? Let's lay it down in a few steps:

1. **Invocation**: When a client calls POST /comment, Quarkus invokes categorize.categorize(...).
2. **Prompt assembly**: LangChain4j concatenates the system message and the filled-in user message, wrapping the comment text in the --- delimiters.
3. **LLM call**: The extension sends the prompt to your configured provider (OpenAI, Hugging Face, etc.) over HTTP.
4. **JSON parse**: The raw model output is parsed back into a CommentReview instance via Jackson.
5. **Response**: You return a fully typed Java record—no manual JSON handling required.

By externalizing your prompts in annotations, you keep your Java code concise and declarative. You get full type-safety on inputs and outputs, easy reuse of prompt templates, and the ability to version control your prompt logic alongside the rest of your application. This is the essence of *prompt engineering* in Quarkus: building AI behavior directly into your codebase in a transparent, maintainable way.

Using Tools in Quarkus

That earlier example is fairly basic. In a production scenario, you'll want to enrich the LLM's capabilities with external tools and document stores. The @RegisterAiService annotation lets you specify exactly which tools and stores to integrate. When your AI service needs live, domain-specific data—like customer names, inventory levels, or product details—you can expose any CDI bean method as a "tool" for the model to call. LangChain4j will handle wiring, invocation, and result injection for you.

Tools are methods that the LLM can call. To define a tool, annotate a bean method with @Tool.

In the previous example, you leverage the Panache repository pattern to interact with the database. We've marked a specific bean method with @Tool to fetch a customer's details, so whenever the LLM needs that information, Quarkus invokes the method and returns the result. In following sections, you will integrate information from the database that can be invoked and fetched using the Tools utility, as displayed in Figure 12-3.

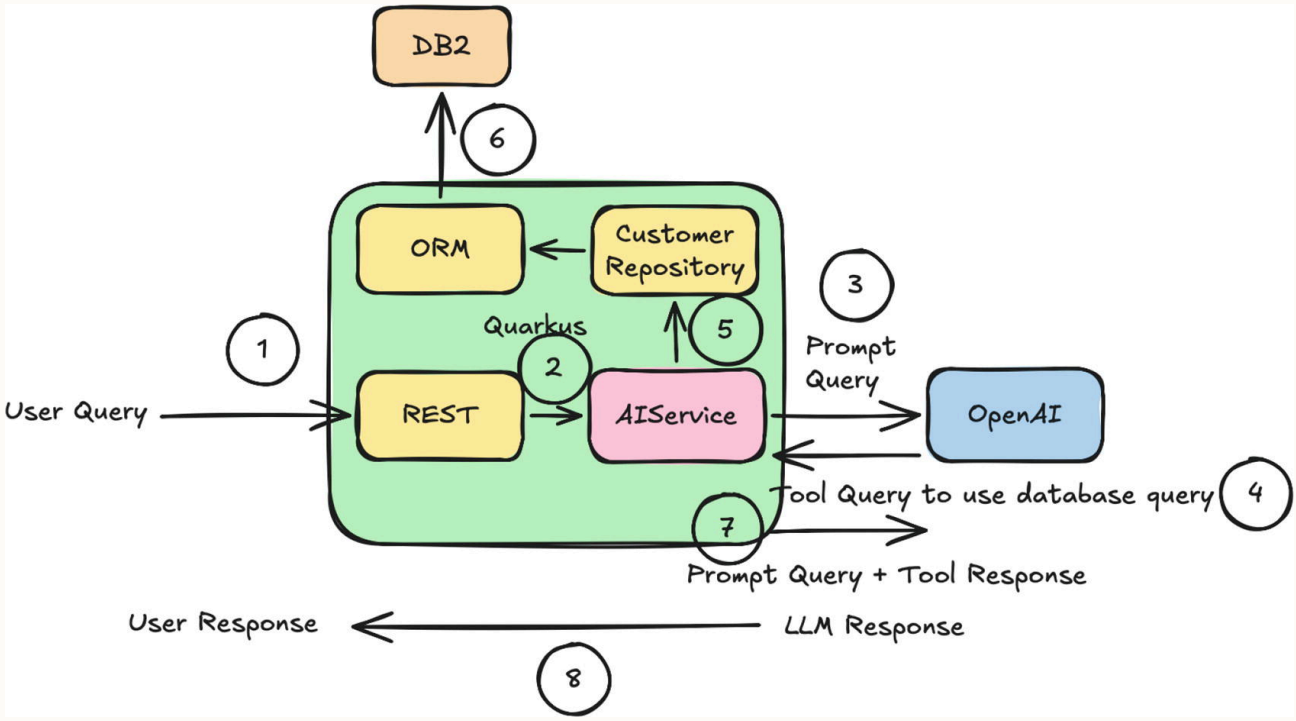


Figure 12-3 Quarkus LangChain4j flows with tools integration

Since you don't want to expose every operation to the model, you also have to enumerate the permitted tools within the @RegisterAiService annotation.

You will start by creating the Customer Entity class, which represents the database table for the customer data. It extends PanacheEntity for simplified JPA functionality with Quarkus. The entity contains some basic fields like name, address, and a unique primary key called id.

File: src/main/java/io/demo/langchain4j/sample/dto/Customer.java

```
package io.demo.langchain4j.sample.dto;
import io.quarkus.hibernate.orm.panache.PanacheEntity;
import jakarta.persistence.Entity;
// 1. Customer Entity
//   - Represents the database table for customer data.
//   - Uses PanacheEntity for simplified JPA with Quarkus.
@Entity
public class Customer extends PanacheEntity {
    public String name;
```

```
    public String address;
    // Add constructors, getters, and setters as needed.
   public Customer() {
    }
    public Customer(String name, String address) {
       this.name = name;
       this.address = address;
    }
    public Long getId() {
       return id;
    }
    public void setId(Long id) {
       this.id = id;
    }
    public String getName() {
       return name;
    }
    public void setName(String name) {
       this.name = name;
    }
    public String getAddress() {
       return address;
    }
    public void setAddress(String address) {
       this.address = address;
    }
}
```

Now you will need to create a CustomerRepository class that extends the PanacheRepository to handle basic CRUD activities along with custom-defined methods. In your case, you have additionally defined a method named getCustomerDetail that takes an id parameter as an argument. This method has been annotated with the @Tool annotation to indicate to Quarkus and LangChain4j that this method is available as a tool. The @Tool annotation takes as an argument the description of the tool. This is important and helps the LLM determine whether this tool can be used for any of the tasks that might be relevant to the customer's request.

File: src/main/java/io/demo/langchain4j/sample/dto/CustomerRepository.java

```
package io.demo.langchain4j.sample.dto;
import dev.langchain4j.agent.tool.Tool;
import io.quarkus.hibernate.orm.panache.PanacheRepository;
import jakarta.enterprise.context.ApplicationScoped;
// Extends PanacheRepository to handle CRUD.
```

```
@ApplicationScoped
public class CustomerRepository implements PanacheRepository<Customer> {
   // The @Tool annotation on getCustomerDetail makes this method
  // callable by the model. The string description helps the model
  // choose the right tool.
  @Tool("get the customer detail for the given customerId")
   public String getCustomerDetail(long id) {
     // Implements a custom method to get the customer detail by ID.
     return find("id", id).firstResult().name;
   }
}
```

You have a tool now available that fetches customer names from the database based on id. Let's provide this tool to the @RegisterAiService. In the following code change, you can observe that we are making changes to the annotation @RegisterAiService by providing it with a new parameter called tools and assigning the CustomerRepository.class class. Additionally, you change the system prompt to indicate to the LLM that in its greeting it needs to include the customer's name if it has it available. This will guide it to search for the customer's details using the tool box. This is also made explicit in the user prompt by instructing the LLM to find the customer name if there is an id available to it. Finally, there is an additional argument field called customerId available to the prompt template that will be provided by the controller layer.

File: src/main/java/io/demo/langchain4j/sample/service/SupportService.java

```
package io.demo.langchain4j.sample.service;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import io.demo.langchain4j.sample.dto.CustomerRepository;
import io.quarkiverse.langchain4j.RegisterAiService;
@RegisterAiService(
      // Declares that CustomerRepository is available as a tool. LangChain4J will scan its
      // @Tool methods and register them with the LLM client.
      tools = {CustomerRepository.class}
)
public interface SupportService {
   @SystemMessage("""
      You are working for an ecommerce company, processing comments
      about their products and providing additional information as
      requested by the customer. Categorise comments into positive
      neutral and negative ones, responding with a JSON format
      document that includes a personalized greeting if you have the
      customer's details.
```

```
    """)
    @UserMessage("""
            Your task is to process the comment delimited by ---.
            Apply sentiment analysis to the comment to determine
            if it is positive, neutral or negative.
            For example:
            - `I love the user experience in your app, you are my best place to be!` is a
'POSITIVE'
              comment
            - `J'adore ton application` is a 'POSITIVE' comment
            - `I don't want to use your app, it's too slow!` is a 'NEGATIVE'
              comment
            Respond with a JSON format containing:
            - the 'rating' key set to 'POSITIVE' if the comment is
            positive, 'NEUTRAL' if the comment is neutral, 'NEGATIVE' otherwise
            - the 'message' key set to a message thanking or
            apologizing to the customer. These messages must be polite
            . Find the customer name if
            there is an id provided and use it to greet the customer.
            ---
            {comment}
            {customerId}
            ---
    """)
    CommentReview categorize(String comment, long customerId);
}
```

When working with RESTful APIs you know you always have a session or contextual user data that helps to identify each user request uniquely. To keep it simple, we have chosen to hard-code the id value to 1 here. This record will be inserted when the application boots up.

File: src/main/java/io/demo/langchain4j/sample/controller/CommentResource.java

```
package io.demo.langchain4j.sample.controller;
import io.demo.langchain4j.sample.service.Comment;
import io.demo.langchain4j.sample.service.SupportService;
import io.demo.langchain4j.sample.service.CommentReview;
import jakarta.inject.Inject;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
@Path("/comment")
public class CommentResource {
    @Inject
    SupportService categorize;
```

```
    @POST
    public CommentReview categorize(Comment comment) {
        // Customer id is hard coded here for simplicity sake.
        return categorize.categorize(comment.comment(), 1);
    }
}
```

The customer id 1 is not available to you yet. Yet again for simplicity, you will insert a database record of a customer when the application boots up through the use of a startup event.

File: src/main/java/io/demo/langchain4j/sample/utils/ApplicationStartup.java

```
package io.demo.langchain4j.sample.utils;
import dev.langchain4j.data.document.Document;
import io.demo.langchain4j.sample.dto.Customer;
import io.quarkus.runtime.StartupEvent;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.event.Observes;
import jakarta.inject.Inject;
import jakarta.transaction.Transactional;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.io.File;
import java.io.IOException;
import java.net.URL;
import java.util.ArrayList;
import java.util.List;
@ApplicationScoped
public class ApplicationStartup {
    private static final Logger LOGGER =
LoggerFactory.getLogger(ApplicationStartup.class);
    @Transactional
    void onStartup(@Observes StartupEvent event) {
        LOGGER.info("Application starting...");
        // 1. Insert a customer into the database
        insertInitialCustomer();
    }
    private void insertInitialCustomer() {
        LOGGER.info("Inserting initial customer...");
        // Inserts a sample customer so that invoking
        // getCustomerDetail(1) returns "John Doe."
        Customer customer = new Customer("John Doe", "123 Main St");
        customer.persist();
```

```
        LOGGER.info("Initial customer inserted with ID: {}", customer.id);
    }
}
```

Let's boot up your application and see the system logs for it. You might see some initial error logs where the system is trying to drop the database and tables that are not present already. Please ignore them for the time being.

```
2025-04-23 09:16:09,277
INFO  [io.qua.dat.dep.dev.DevServicesDatasourceProcessor] (build-49) Dev Services
for default datasource (db2) started - container ID is 165c6f084e07
2025-04-23 09:16:09,281
INFO  [io.qua.hib.orm.dep.dev.HibernateOrmDevServicesProcessor] (build-9) Setting
quarkus.hibernate-orm.database.generation=drop-and-create to initialize Dev
Services managed database
2025-04-23 09:16:13,675 INFO  [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus
Main Thread) Application starting...
2025-04-23 09:16:13,675 INFO  [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus
Main Thread) Inserting initial customer...
2025-04-23 09:16:13,832 INFO  [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus
Main Thread) Initial customer inserted with ID: 1
```

Let's now test the curl POST request and see the output from the application. In the following code, you can see your customer name in the output, so the LLM has been able to successfully request that the tool fetch the customer name and then use that information to augment its response to make the message more personal for your customers.

```
curl localhost:8080/comment --header 'Content
-Type: text/plain' --data 'This is a great product! I highly recommend it.'
```

Output:

```
{"rating":"POSITIVE","message":"Thank you for your kind words, John Doe! We're
thrilled to hear you love our product!"}
```

Also, let's see what the system is up to during this time. In the following code, you can see in its first request to OpenAI the prompt is enriched, and the tooling list now contains your database request feature.

```
2025-04-23 09:18:14,506
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-
```

thread-1) Request:
- method: POST
- url: https://api.openai.com/v1/chat/completions
- headers: [Accept: application/json], [Authorization: Be...kA], [Content-Type: application/json], [User-Agent: langchain4j-openai], [content-length: 1959]
- body: {
  "model" : "gpt-4o-mini",
  "messages" : [{
    "role" : "system",
    "content" : "   You are working for an ecommerce company, processing comments about\n   their products. Categorize comments into positive, neutral and negative ones,\nresponding with a JSON format document that includes a\n   personalized greeting if you have the customer's details.\n"
  }, {
    "role" : "user",
    "content" : "   Your task is to process the comment delimited by ---.\n   Apply sentiment analysis to the comment to determine\n   if it is positive, neutral or negative.\n\n   For example:\n   - `I love the user experience in your app, you are my best place to be!` is a 'POSITIVE' comment\n   - `J'adore ton application` is a 'POSITIVE' comment\n   - `I don't want to use your app, it's too slow!` is a 'NEGATIVE' comment\n\n   Respond with a JSON format containing:\n   - the 'rating' key set to 'POSITIVE' if the comment is\n   positive, 'NEUTRAL' if the comment is neutral, 'NEGATIVE' otherwise\n   - the 'message' key set to a message thanking or apologizing\n   to the customer. These messages must be polite. Find the customer name if there is an id provided\n   and use it to greet the customer.\n\n   ---\n   This is a great product! I highly recommend it.\n   1\n   ---\n\nYou must answer strictly in the following JSON format: {\n\"rating\": (type: enum, must be one of [POSITIVE, NEUTRAL, NEGATIVE]),\n\"message\": (type: string)\n}"
  }],
  "temperature" : 0.5,
  "top_p" : 1.0,
  "presence_penalty" : 0.0,
  "frequency_penalty" : 0.0,
  "tools" : [{
    "type" : "function",
    "function" : {
      "name" : "getCustomerDetail",
      "description" : "get the customer detail for the given customerId",
      "parameters" : {
        "type" : "object",
        "properties" : {
          "id" : {
            "type" : "integer"
          }
        },

```
      "required" : [ "id" ]
     }
    }
   } ]
}
```

In its response, OpenAI has suggested to use the tool getCustomerDetail with argument 1, which will then be invoked by AIService.

```
2025-04-23 09:18:15,478
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-thread-1) Response:
- status code: 200
- headers: [Date: Wed, 23 Apr 2025 09:18:15 GMT], [Content-Type: application/json], [Transfer-Encoding: chunked], [Connection: keep-alive], [access-control-expose-headers: X-Request-ID], [openai-organization: user-eldk0kvytvetb2vdgo2y3nvr], [openai-processing-ms: 292], [openai-version: 2020-10-01], [x-ratelimit-limit-requests: 10000], [x-ratelimit-limit-tokens: 200000], [x-ratelimit-remaining-requests: 9999], [x-ratelimit-remaining-tokens: 199667], [x-ratelimit-reset-requests: 8.64s], [x-ratelimit-reset-tokens: 99ms], [x-request-id: req_7951806f5d7eaffea1f1a97fb3ad7bdc], [strict-transport-security: max-age=31536000; includeSubDomains; preload], [cf-cache-status: DYNAMIC], [Set-Cookie: __...ne], [X-Content-Type-Options: nosniff], [Set-Cookie: _c...ne], [Server: cloudflare], [CF-RAY: 934c45bca9e95d63-FRA], [alt-svc: h3=":443"; ma=86400]
- body: {
 "id": "chatcmpl-BPQX9E9kxe5Xr5dEFJvtFElpWdNX4",
 "object": "chat.completion",
 "created": 1745399895,
 "model": "gpt-4o-mini-2024-07-18",
 "choices": [
  {
   "index": 0,
   "message": {
    "role": "assistant",
    "content": null,
    "tool_calls": [
     {
      "id": "call_s6MVpXkOTA0TvsjtpdBPdLAb",
      "type": "function",
      "function": {
       "name": "getCustomerDetail",
       "arguments": "{\"id\":1}"
      }
     }
    ],
```

```
      "refusal": null,
      "annotations": []
     },
     "logprobs": null,
     "finish_reason": "tool_calls"
    }
   ],
   "usage": {
    "prompt_tokens": 343,
    "completion_tokens": 16,
    "total_tokens": 359,
    "prompt_tokens_details": {
     "cached_tokens": 0,
     "audio_tokens": 0
    },
    "completion_tokens_details": {
     "reasoning_tokens": 0,
     "audio_tokens": 0,
     "accepted_prediction_tokens": 0,
     "rejected_prediction_tokens": 0
    }
   },
   "service_tier": "default",
   "system_fingerprint": "fp_0392822090"
}
```


The response from the tool call is then sent back to OpenAI with the augmented prompt to provide it with the customer name information, which it uses to create the final response for you.


```
2025-04-23 09:18:16,049
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-thread-1) Request:
- method: POST
- url: https://api.openai.com/v1/chat/completions
- headers: [Accept: application/json], [Authorization: Be...kA], [Content-Type: application/json], [User-Agent: langchain4j-openai], [content-length: 2309]
- body: {
  "model" : "gpt-4o-mini",
  "messages" : [ {
    "role" : "system",
    "content" : "      You are working for an ecommerce company, processing comments about\n      their products. Categorize comments into positive, neutral and negative ones,\nresponding with a JSON format document that includes a\n      personalized greeting if you have the customer's details.\n"
```

```
 }, {
   "role" : "user",
   "content" : "       Your task is to process the comment delimited by ---.\n       Apply
sentiment analysis to the comment to determine\n       if it is positive, neutral or
negative.\n\n       For example:\n       - `I love the user experience in your app, you are
my best place to be!` is a 'POSITIVE' comment\n       - `J'adore ton application` is a
'POSITIVE' comment\n       - `I don't want to use your app, it's too slow!` is a
'NEGATIVE' comment\n\n       Respond with a JSON format containing:\n       - the
'rating' key set to 'POSITIVE' if the comment is\n       positive, 'NEUTRAL' if the
comment is neutral, 'NEGATIVE' otherwise\n       - the 'message' key set to a message
thanking or apologizing\n       to the customer. These messages must be polite. Find
the customer name if there is an id provided\n       and use it to greet the
customer.\n\n       ---\n       This is a great product! I highly recommend
it.\n       1\n       ---\n\nYou must answer strictly in the following JSON format:
{\n\"rating\": (type: enum, must be one of [POSITIVE, NEUTRAL,
NEGATIVE]),\n\"message\": (type: string)\n}"
 }, {
   "role" : "assistant",
   "tool_calls" : [ {
     "id" : "call_s6MVpXkOTAOTvsjtpdBPdLAb",
     "type" : "function",
     "function" : {
       "name" : "getCustomerDetail",
       "arguments" : "{\"id\":1}"
     }
   } ]
 }, {
   "role" : "tool",
   "tool_call_id" : "call_s6MVpXkOTAOTvsjtpdBPdLAb",
   "content" : "\"John Doe\""
 } ],
 "temperature" : 0.5,
 "top_p" : 1.0,
 "presence_penalty" : 0.0,
 "frequency_penalty" : 0.0,
 "tools" : [ {
   "type" : "function",
   "function" : {
     "name" : "getCustomerDetail",
     "description" : "get the customer detail for the given customerId",
     "parameters" : {
       "type" : "object",
       "properties" : {
         "id" : {
           "type" : "integer"
         }
```

```
      },
      "required" : [ "id" ]
    }
   }
  } ]
}
2025-04-23 09:18:17,196
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-thread-1) Response:
- status code: 200
- headers: [Date: Wed, 23 Apr 2025 09:18:17 GMT], [Content-Type: application/json], [Transfer-Encoding: chunked], [Connection: keep-alive], [access-control-expose-headers: X-Request-ID], [openai-organization: user-eldk0kvytvetb2vdgo2y3nvr], [openai-processing-ms: 898], [openai-version: 2020-10-01], [x-ratelimit-limit-requests: 10000], [x-ratelimit-limit-tokens: 200000], [x-ratelimit-remaining-requests: 9998], [x-ratelimit-remaining-tokens: 199662], [x-ratelimit-reset-requests: 16.187s], [x-ratelimit-reset-tokens: 101ms], [x-request-id: req_a46c1fa7cc70ab911321b0e584e70f27], [strict-transport-security: max-age=31536000; includeSubDomains; preload], [cf-cache-status: DYNAMIC], [Set-Cookie: __...ne], [X-Content-Type-Options: nosniff], [Set-Cookie: _c...ne], [Server: cloudflare], [CF-RAY: 934c45c659be5d63-FRA], [alt-svc: h3=":443"; ma=86400]
- body: {
  "id": "chatcmpl-BPQXAe0hFkK0KGf8oIavBUjUGkZhG",
  "object": "chat.completion",
  "created": 1745399896,
  "model": "gpt-4o-mini-2024-07-18",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "{\n  \"rating\": \"POSITIVE\",\n  \"message\": \"Thank you for your kind words, John Doe! We're thrilled to hear you love our product!\"\n}",
        "refusal": null,
        "annotations": []
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 371,
    "completion_tokens": 36,
    "total_tokens": 407,
    "prompt_tokens_details": {
      "cached_tokens": 0,
```

```
      "audio_tokens": 0
    },
    "completion_tokens_details": {
      "reasoning_tokens": 0,
      "audio_tokens": 0,
      "accepted_prediction_tokens": 0,
      "rejected_prediction_tokens": 0
    }
  },
  "service_tier": "default",
  "system_fingerprint": "fp_0392822090"
}
```


The system outputs are helpful to see what is happening behind the curtains.

So to finalize, what happens at runtime?

1. Seeding the database that inserts a sample customer so that invoking getCustomerDetail(1) returns "John Doe."
2. When the LLM prompt runs, LangChain4j will detect the need to fetch {customerId} tool data, call CustomerRepository.getCustomerDetail(1), and include "John Doe" in the prompt context.
3. The final JSON response is parsed into CommentReview, complete with a personalized greeting.

By declaring your existing Quarkus beans as tools, you empower the LLM to execute real business logic—database lookups, external API calls, or any Java method—right in the middle of its reasoning, all with zero manual HTTP or JSON plumbing.

Using ChatMemory in Quarkus

In many AI scenarios—chatbots, multiturn assistants, or comment categorize workflows—you need to preserve conversational context across requests. Chat memory lets your LLM "remember" prior exchanges, making interactions feel more natural and allowing follow-up questions, clarifications, or incremental refinements.

Let's review an example that showcases the importance of in-memory chat history in a conversation with an LLM. The idea is to show how continuity helps the assistant understand context, remember preferences, and respond naturally—just like a human would in a longer chat.



Conversation without in-memory history:

- **User:** What's the capital of France?
- **Assistant:** The capital of France is Paris.
- **User:** And what's the population there?
- **Assistant:** Could you clarify which city you're referring to?

(Without memory, the assistant doesn't know "there" means Paris.)

In the previous conversation each question-answer pair acts as a separate independent session, and the LLM does not have any context of the previous history. This is the reason it is not able to infer the city for the second query. This is illustrated in Figure 12-4.

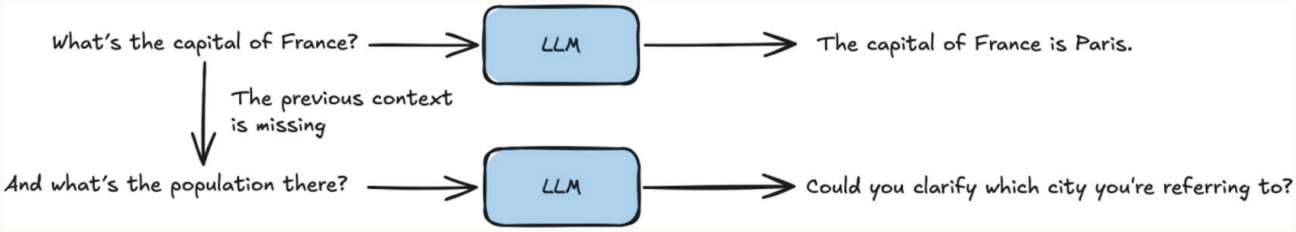


Figure 12-4 Conversational flow without context to previous conversation history



Conversation with in-memory chat history:

- **User:** What's the capital of France?
- **Assistant:** The capital of France is Paris.
- **User:** And what's the population there?
- **Assistant:** As of 2024, Paris has a population of about 2.1 million people.

(The assistant remembers "Paris" from the previous message.)

As you can see, with in-context memory from previous chat history, the LLM now has a context of previous conversation and can gain insight related to any contextual question the user might have and that is not explicitly repeated in the latest question.

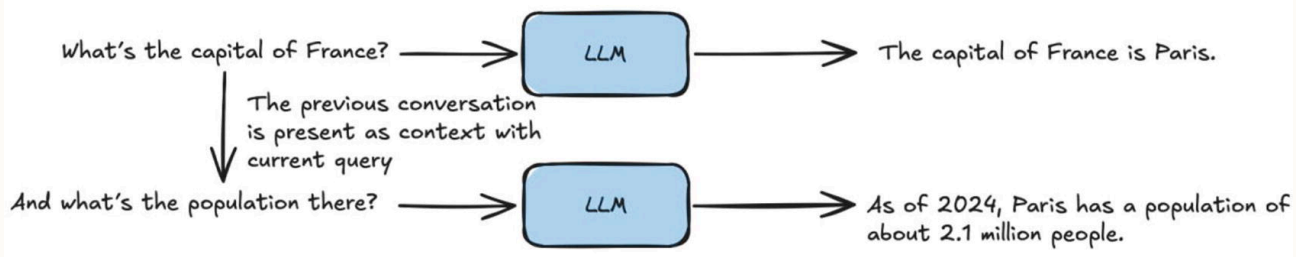


Figure 12-5 Conversational flow with context to previous conversation history

✅ Why this matters:

In-memory chat history helps the assistant:

1. Understand pronouns like "there" or "he/she/it"
2. Follow a topic across multiple turns (like planning a trip or solving a coding issue)
3. Keep user preferences in mind ("I'm vegan" → later recommends vegan restaurants)

Defining a ChatMemoryProvider

Because LLMs don't retain state, you must supply the interaction context ("memory") with each request. To keep that context from growing unbounded, older messages need to be pruned. You can customize which ChatMemoryProvider is used by setting the chatMemoryProviderSupplier property on the @RegisterAiService annotation. By default, it's set to RegisterAiService.BeanChatMemoryProviderSupplier.class, which causes the AiService to pick up whichever ChatMemoryProvider bean your application defines—or fall back to the extension's built-in implementation.

We will create an in-memory chat store for storing your conversations. The conversations won't be persistent after a reboot. For production, you can use Redis or any similar memory performant database. Create a bean named ChatMemoryBean that implements the ChatMemoryProvider interface. Then initialize an instance of InMemoryChatMemoryStore and override the get method, which returns a sliding window memory storing a maximum of 20 messages based on memoryId.

File: src/main/java/io/demo/langchain4j/sample/utils/ChatMemoryBean.java

```
package io.demo.langchain4j.sample.utils;
import java.util.function.Supplier;
import dev.langchain4j.memory.ChatMemory;
import dev.langchain4j.memory.chat.ChatMemoryProvider;
import dev.langchain4j.memory.chat.MessageWindowChatMemory;
import dev.langchain4j.store.memory.chat.InMemoryChatMemoryStore;
import jakarta.enterprise.context.ApplicationScoped;
@ApplicationScoped
public class ChatMemoryBean implements Supplier<ChatMemoryProvider> {
   // A simple, JVM-local store that holds all messages in memory.
  // For production, you could swap in a Redis or database-backed
  // store.
  private final InMemoryChatMemoryStore store = new InMemoryChatMemoryStore();
   @Override
   public ChatMemoryProvider get() {
      return new ChatMemoryProvider() {
```

```
        @Override
        public ChatMemory get(Object memoryId) {
          // A sliding window memory: it retains up to
          // maxMessages most recent turns, dropping the oldest
          // as new ones arrive.
          return MessageWindowChatMemory.builder()
                  .maxMessages(20)
                  // A unique key (e.g. session ID, user ID, or
                  // review ID) that scopes separate
                  // conversations.
                  .id(memoryId)
                  .chatMemoryStore(store)
                  .build();
        }
      };
    }
}
```

Enabling Memory in Your AI Service

Once the in-memory store bean is set up, you can use it inside your @RegisterAiService annotation by passing it as an argument to chatMemoryProviderSupplier. You also pass some contextual information about some products you sell and to advise the LLM to answer any additional information requested by your customer.

File: src/main/java/io/demo/langchain4j/sample/service/SupportService.java

```
package io.demo.langchain4j.sample.service;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import io.demo.langchain4j.sample.dto.CustomerRepository;
import io.demo.langchain4j.sample.rag.RetrievalAugmentorExample;
import io.demo.langchain4j.sample.utils.ChatMemoryBean;
import io.quarkiverse.langchain4j.RegisterAiService;
@RegisterAiService(
      tools = {CustomerRepository.class},
      chatMemoryProviderSupplier = ChatMemoryBean.class
)
public interface SupportService {
   @SystemMessage("""
     You are working for an ecommerce company, processing comments
     about their products and providing additional information as
     requested by the customer. Categorize comments into positive,
     neutral and negative ones, responding with a JSON format that
```

```
        includes a personalized greeting if you have the customer's
        name.
        Products we sell:
        ProductID,Name,Description,Price,Category,Manufacturer,SKU
       1004,Laptop,15-inch laptop with SSD,999.00,Electronics,BrandD,LP-1004
       1005,Coffee Maker,Automatic coffee maker,49.99,Appliances,BrandE,CM-1005
    """)
    @UserMessage("""
          Your task is to process the comment delimited by ---.
          Apply sentiment analysis to the comment to determine
          if it is positive, neutral or negative.
          For example:
          - `I love the user experience in your app, you are my best place to be!` is a
'POSITIVE'
          comment
          - `J'adore ton application` is a 'POSITIVE' comment
          - `I don't want to use your app, it's too slow!` is a 'NEGATIVE'
          comment
          Respond with a JSON format containing:
          - the 'rating' key set to 'POSITIVE' if the comment is
          positive, 'NEUTRAL' if the comment is neutral, 'NEGATIVE' otherwise
          - the 'message' key set to a message thanking or
          apologizing to the customer. These messages must be polite
          . Find the customer name if
          there is an id provided and use it to greet the customer.
          Also provide any additional information as requested by
          the customer.
          ---
          {comment}
          {customerId}
          ---
    """)
    CommentReview categorize(String comment, long customerId);
}
```

This simple change should enable you to establish a contextual history conversation by setting up a chat memory store. Let's boot up your application and send a few curl POST requests. First you will request info for all the products.

```
curl localhost:8080/comment --header 'Content-Type: text/plain' --data 'This is a
great product! I highly recommend it. Could you list all products from your shop
catalog?'
```

Output:

{"rating":"POSITIVE","message":"Hello John Doe, thank you for your wonderful comment! We're glad to hear you love the product. Here are all the products from our shop catalog: [{\"ProductID\":1004,\"Name\":\"Laptop\",\"Description\":\"15-inch laptop with SSD\",\"Price\":999.00,\"Category\":\"Electronics\",\"Manufacturer\":\"BrandD\",\"SKU\":\"LP-1004\"},{\"ProductID\":1005,\"Name\":\"Coffee Maker\",\"Description\":\"Automatic coffee maker\",\"Price\":49.99,\"Category\":\"Appliances\",\"Manufacturer\":\"BrandE\",\"SKU\":\"CM-1005\"}]."}

```
curl localhost:8080/comment --header 'Content-Type: text/plain' --data 'I had previously purchased Laptop - 15-inch laptop with SSD. This is a great product! I highly recommend it.'
```

Output:

{"rating":"POSITIVE","message":"Dear John Doe, thank you for your wonderful comment! We're thrilled to hear that you love the Laptop. If you have any questions or need further assistance, feel free to ask!"}

```
curl localhost:8080/comment --header 'Content-Type: text/plain' --data 'Yes, it was a great product! Could you please tell me the SKU for it?'
```

Output:

{"rating":"POSITIVE","message":"Hello John Doe, thank you for your positive feedback! The SKU for the product is TS-1001."}

You can see that it has the contextual information from previous conversations to answer your last customer query. Let's also review some system logs to see what's happening on the application backend.

First request log:

```
{
  "role" : "system",
  "content" : "    You are working for an ecommerce company, processing comments about\n    their products and providing additional information as requested\n   \tby the customer. Categorize comments into positive, neutral and negative ones,\n    responding with a JSON format document that includes a\n    personalized greeting if
```

you have the customer's details.\n Products we
sell:\n ProductID,Name,Description,Price,Category,Manufacturer,SKU\n 1004,Lapt
op,15-inch laptop with SSD,999.00,Electronics,BrandD,LP-1004\n 1005,Coffee
Maker,Automatic coffee maker,49.99,Appliances,BrandE,CM-1005\n"
}, {
"role" : "user",
"content" : " Your task is to process the comment delimited by ---.\n Apply
sentiment analysis to the comment to determine\n if it is positive, neutral or
negative.\n\n For example:\n - `I love the user experience in your app, you are
my best place to be!` is a 'POSITIVE' comment\n - `J'adore ton application` is a
'POSITIVE' comment\n - `I don't want to use your app, it's too slow!` is a
'NEGATIVE' comment\n\n Respond with a JSON format containing:\n - the
'rating' key set to 'POSITIVE' if the comment is\n positive, 'NEUTRAL' if the
comment is neutral, 'NEGATIVE' otherwise\n - the 'message' key set to a message
thanking or apologizing\n to the customer. These messages must be polite. Find
the customer name if there is an id provided\n and use it to greet the customer.
Also provide any additional\n information as requested by the customer.\n\n ---
\n **This is a great product! I highly recommend it. Could you list all products from
your shop catalog?**\n 1\n ---\n\nYou must answer strictly in the following
JSON format: {\n\"rating\": (type: enum, must be one of [POSITIVE, NEUTRAL,
NEGATIVE]),\n\"message\": (type: string)\n}"
}

Second request log:

{
"role" : "system",
"content" : " You are working for an ecommerce company, processing comments
about\n their products and providing additional information as requested\n \tby the
customer. Categorize comments into positive, neutral and negative ones,\n
responding with a JSON format document that includes a\n personalized greeting if
you have the customer's details.\n Products we
sell:\n ProductID,Name,Description,Price,Category,Manufacturer,SKU\n 1004,Lapt
op,15-inch laptop with SSD,999.00,Electronics,BrandD,LP-1004\n 1005,Coffee
Maker,Automatic coffee maker,49.99,Appliances,BrandE,CM-1005\n"
}, {
"role" : "user",
"content" : " Your task is to process the comment delimited by ---.\n Apply
sentiment analysis to the comment to determine\n if it is positive, neutral or
negative.\n\n For example:\n - `I love the user experience in your app, you are
my best place to be!` is a 'POSITIVE' comment\n - `J'adore ton application` is a
'POSITIVE' comment\n - `I don't want to use your app, it's too slow!` is a
'NEGATIVE' comment\n\n Respond with a JSON format containing:\n - the
'rating' key set to 'POSITIVE' if the comment is\n positive, 'NEUTRAL' if the
comment is neutral, 'NEGATIVE' otherwise\n - the 'message' key set to a message

thanking or apologizing\n to the customer. These messages must be polite. Find the customer name if there is an id provided\n and use it to greet the customer. Also provide any additional\n information as requested by the customer.\n\n ---\n **I had previously purchased Laptop - 15-inch laptop with SSD. This is a great product! I highly recommend it.**\n 1\n ---\n\nYou must answer strictly in the following JSON format: {\n\"rating\": (type: enum, must be one of [POSITIVE, NEUTRAL, NEGATIVE]),\n\"message\": (type: string)\n}"
}

Third request log:

{
"role" : "system",
"content" : " You are working for an ecommerce company, processing comments about\n their products and providing additional information as requested\n \tby the customer. Categorize comments into positive, neutral and negative ones,\n responding with a JSON format document that includes a\n personalized greeting if you have the customer's details.\n Products we sell:\n ProductID,Name,Description,Price,Category,Manufacturer,SKU\n 1004,Laptop,15-inch laptop with SSD,999.00,Electronics,BrandD,LP-1004\n 1005,Coffee Maker,Automatic coffee maker,49.99,Appliances,BrandE,CM-1005\n"
}, {
"role" : "user",
"content" : " Your task is to process the comment delimited by ---.\n Apply sentiment analysis to the comment to determine\n if it is positive, neutral or negative.\n\n For example:\n - `I love the user experience in your app, you are my best place to be!` is a 'POSITIVE' comment\n - `J'adore ton application` is a 'POSITIVE' comment\n - `I don't want to use your app, it's too slow!` is a 'NEGATIVE' comment\n\n Respond with a JSON format containing:\n - the 'rating' key set to 'POSITIVE' if the comment is\n positive, 'NEUTRAL' if the comment is neutral, 'NEGATIVE' otherwise\n - the 'message' key set to a message thanking or apologizing\n to the customer. These messages must be polite. Find the customer name if there is an id provided\n and use it to greet the customer. Also provide any additional\n information as requested by the customer.\n\n ---\n **I had previously purchased Laptop - 15-inch laptop with SSD. This is a great product! I highly recommend it.**\n 1\n ---\n\nYou must answer strictly in the following JSON format: {\n\"rating\": (type: enum, must be one of [POSITIVE, NEUTRAL, NEGATIVE]),\n\"message\": (type: string)\n}"
}

So what happens during the runtime?

- **chatMemoryProviderSupplier = ChatMemoryBean.class**

Instructs the Quarkus LangChain4u extension to use your ChatMemoryBean to obtain a ChatMemoryProvider.

- On each call to categorize(...), LangChain4j will:
  1. Fetch or create a ChatMemory for the given memory ID.
  2. Replay the last N messages (system, user, assistant) before sending the new prompt.
  3. Append the latest user input and model response back into the memory store.

  Runtime behavior:

  1. **First request**: The memory is empty, so only the system and user messages go to the LLM.
  2. **Subsequent requests**: The previous turns (up to 20) are prefixed to the prompt, giving the model access to earlier context.
  3. **Stateless API, stateful experience**: Your REST endpoint remains a simple POST, but users get coherent, multiturn conversations.

By wiring in a ChatMemoryProvider, your Quarkus application gains stateful dialogue capabilities without cluttering your business logic. You simply define how memory is stored and let LangChain4j handle the rest.

Using the RAG Pattern in Quarkus

RAG combines the creative power of LLMs with a grounding in your own data. Instead of relying solely on the model's pretrained knowledge, you ingest domain documents into a vector store, retrieve the most semantically relevant passages at query time, and feed those passages into your prompt—ensuring up-to-date, factual responses. Figure 12-6 shows this flow.

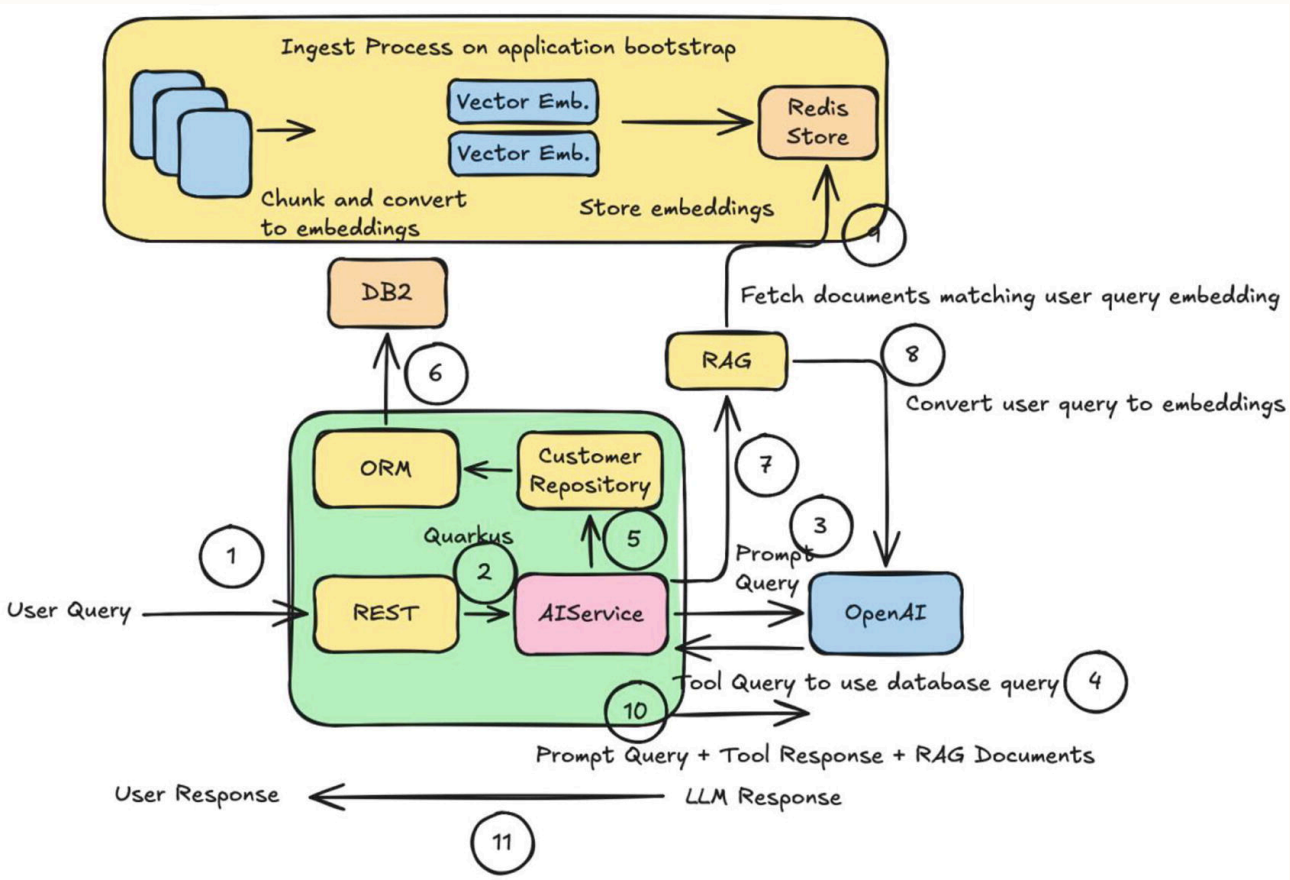


Figure 12-6 RAG flow using LangChain4j in Quarkus

There are two steps to this flow:

1. Ingest documents into your Redis vector store.
2. Use RAG to fetch documents that could be leveraged by LLM to answer user queries.

Ingest Documents into Your Redis Vector Store

Let's focus first on ingesting documents. You have a sample product catalog document that the customer could request information about. It is a CSV file with the first row containing the column/field names and the subsequent rows containing each product in the catalog. The fields are product ID, name, description, price, category, manufacturer, and SKU. You will store this file in the resource directory.

File: src/main/resources/catalog.csv

```
ProductID,Name,Description,Price,Category,Manufacturer,SKU
1001,T-Shirt,Comfortable cotton t-shirt,15.99,Clothing,BrandA,TS-1001
1002,Jeans,Classic blue denim jeans,39.99,Clothing,BrandB,JN-1002
1003,Running Shoes,Lightweight running shoes,79.99,Shoes,BrandC,RS-1003
1004,Laptop,15-inch laptop with SSD,999.00,Electronics,BrandD,LP-1004
```

```
1005,Coffee Maker,Automatic coffee maker,49.99,Appliances,BrandE,CM-1005
```

As shown in Figure 12-6, you need to ingest these files so that they are ready for querying from your RAG pipeline. To enable this, you will ingest them when your application bootstraps. In the ApplicationStartup.java file, you will enable this functionality. Before that, you will create a few helper classes to modularize some of these tasks.

You will create a class called DocumentFromTextCreationExample, which takes as an argument a file and converts it to the LangChain4j document format by first parsing the text from the file document and creating the Document instance. The Document instance is a standard format for LangChain4j and can be used with the Redis vector store.

File: src/main/java/io/demo/langchain4j/sample/utils/DocumentFromTextCreationExample.java

```
package io.demo.langchain4j.sample.utils;
import static
dev.langchain4j.data.document.loader.FileSystemDocumentLoader.loadDocument;
import java.io.File;
import dev.langchain4j.data.document.Document;
import dev.langchain4j.data.document.parser.TextDocumentParser;
import jakarta.enterprise.context.ApplicationScoped;
@ApplicationScoped
public class DocumentFromTextCreationExample {
   Document createDocument(File file) {
      // Wraps FileSystemDocumentLoader + TextDocumentParser to turn
      // catalog.csv into a Document.
      return loadDocument(file.toPath(), new TextDocumentParser());
   }
}
```

The next taskis to create a helper class for ingesting a list of documents into the Redis vector store. This includes two tasks. First, convert the document to embeddings by making a call to the OpenAI Embeddings model and then storing the vector by embedding it in a Redis store. The ingest method takes a list of document types and creates an EmbeddingStoreIngestor using the store and embedding model provided at runtime. It then calls documentSplitter, which breaks the documents into smaller chunks so they fit within the LLM's prompt size limits.

File: src/main/java/io/demo/langchain4j/sample/utils/IngestorExample.java

```
package io.demo.langchain4j.sample.utils;
import static dev.langchain4j.data.document.splitter.DocumentSplitters.recursive;
import java.util.List;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import dev.langchain4j.data.document.Document;
import dev.langchain4j.model.embedding.EmbeddingModel;
import dev.langchain4j.store.embedding.EmbeddingStoreIngestor;
import io.quarkiverse.langchain4j.redis.RedisEmbeddingStore;
// Uses a Quarkus-provided RedisEmbeddingStore and an EmbeddingModel
// (e.g., via OpenAI) to split each document into chunks and store
// their embeddings.
@ApplicationScoped
public class IngestorExample {
   /**
    * The embedding store (the database).
   * The bean is provided by the quarkus-langchain4j-redis extension.
   */
   @Inject
   RedisEmbeddingStore store;
   /**
    * The embedding model (how is computed the vector of a document).
   * The bean is provided by the LLM (like openai) extension.
   */
   @Inject
   EmbeddingModel embeddingModel;
   public void ingest(List<Document> documents) {
      EmbeddingStoreIngestor ingestor = EmbeddingStoreIngestor.builder()
            .embeddingStore(store)
            .embeddingModel(embeddingModel)
            .documentSplitter(recursive(500, 0))
            .build();
      // Warning - this can take some time...
     ingestor.ingest(documents);
   }
}
```

Now you can use these helper classes, which are injected at run time, to ingest your document into the Redis vector store. First convert your file content to Document format using the following helper method. This method loads the file from the resource directory and uses the createDocument method of the DocumentFromTextCreationExample class to convert it to Document format and store it in a list.

```
private List<Document> readDocumentsFromCatalogCsv() throws IOException {
```

```
    // Load the file from the resources folder.
    URL resource = getClass().getClassLoader().getResource("catalog.csv");
    if (resource == null) {
        throw new IOException("File catalog.csv not found in resources folder");
    }
    File file = new File(resource.getFile());
    Document document = documentFromTextCreationExample.createDocument(file);
    List<Document> documents = new ArrayList<>();
    documents.add(document);
    LOGGER.info("Documents read from catalog.csv");
    return documents;
}
```

Then using the ingestDocumentsIntoRedis method, you ingest the documents into Redis through the previously created IngestorExample class by calling the ingest method.

```
private void ingestDocumentsIntoRedis(List<Document> documents) {
    LOGGER.info("Ingesting documents into Redis...");
    ingestorExample.ingest(documents);
    LOGGER.info("Documents ingested into Redis.");
}
```

The complete code is as follows:

File: src/main/java/io/demo/langchain4j/sample/utils/ApplicationStartup.java

```
package io.demo.langchain4j.sample.utils;
import dev.langchain4j.data.document.Document;
import io.demo.langchain4j.sample.dto.Customer;
import io.quarkus.runtime.StartupEvent;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.event.Observes;
import jakarta.inject.Inject;
import jakarta.transaction.Transactional;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.io.File;
import java.io.IOException;
import java.net.URL;
import java.util.ArrayList;
import java.util.List;
@ApplicationScoped
public class ApplicationStartup {
```

```
    private static final Logger LOGGER =
LoggerFactory.getLogger(ApplicationStartup.class);
    @Inject
    IngestorExample ingestorExample;
    @Inject
    DocumentFromTextCreationExample documentFromTextCreationExample;
    @Transactional
    void onStartup(@Observes StartupEvent event) {
        LOGGER.info("Application starting...");
        // 1. Insert a customer into the database
       insertInitialCustomer();
        // 2. Read documents from catalog.csv and ingest into Redis
       try {
            List<Document> documents = readDocumentsFromCatalogCsv();
            ingestDocumentsIntoRedis(documents);
        } catch (IOException e) {
            LOGGER.error("Error processing catalog.csv: {}", e.getMessage(), e);
        }
    }
    private void insertInitialCustomer() {
        LOGGER.info("Inserting initial customer...");
        Customer customer = new Customer("John Doe", "123 Main St");
        customer.persist();
        LOGGER.info("Initial customer inserted with ID: {}", customer.id);
    }
    private List<Document> readDocumentsFromCatalogCsv() throws IOException {
        // Load the file from the resources folder.
       URL resource = getClass().getClassLoader().getResource("catalog.csv");
        if (resource == null) {
            throw new IOException("File catalog.csv not found in resources folder");
        }
        File file = new File(resource.getFile());
        Document document =
documentFromTextCreationExample.createDocument(file);
        List<Document> documents = new ArrayList<>();
        documents.add(document);
        LOGGER.info("Documents read from catalog.csv");
        return documents;
    }
    private void ingestDocumentsIntoRedis(List<Document> documents) {
        LOGGER.info("Ingesting documents into Redis...");
        ingestorExample.ingest(documents);
        LOGGER.info("Documents ingested into Redis.");
    }
}
```

Use RAG to fetch documents that could be leveraged by the LLM to answer user queries.

The next goal is to use the RAG pattern to convert the user query to embeddings and then use the embeddings to fetch semantically similar documents from the Redis vector store.

Define Your RetrievalAugmentor

The first task is to create a bean class that implements the RetrievalAugmentor interface. In the constructor of this class, you will create instances of the retriever and augmentor and finally implement the get method, which returns the augmentor instance.

The retriever instance is an instance of EmbeddingStoreContentRetriever, and using the builder method you pass the embedding model, which the retriever can use to convert user query to embeddings and the embedding store to which it needs to do a similarity search. Through the maxResults method you can pass how many matched similar documents should be returned.

```
EmbeddingStoreContentRetriever contentRetriever =
EmbeddingStoreContentRetriever.builder()
        .embeddingModel(model)
        .embeddingStore(store)
        .maxResults(3)
        .build();
```

Once the retriever object is created, it is passed as a parameter to the DefaultRetrievalAugmentor instance created using the builder method, where the retriever object is passed as an argument to the contentRetriever method. This instantiates your augmentor instance that is returned from the get method of your class.

The complete code for this change is shown here:

File: src/main/java/io/demo/langchain4j/sample/rag/RetrievalAugmentorExample.java

```
package io.demo.langchain4j.sample.rag;
import java.util.function.Supplier;
import jakarta.enterprise.context.ApplicationScoped;
import dev.langchain4j.model.embedding.EmbeddingModel;
import dev.langchain4j.rag.DefaultRetrievalAugmentor;
import dev.langchain4j.rag.RetrievalAugmentor;
import dev.langchain4j.rag.content.retriever.EmbeddingStoreContentRetriever;
import io.quarkiverse.langchain4j.redis.RedisEmbeddingStore;
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
@ApplicationScoped
public class RetrievalAugmentorExample implements Supplier<RetrievalAugmentor> {
   private static final Logger LOGGER =
LoggerFactory.getLogger(RetrievalAugmentorExample.class);
   private final RetrievalAugmentor augmentor;
   RetrievalAugmentorExample(RedisEmbeddingStore store, EmbeddingModel model)
{
      LOGGER.info("Creating RetrievalAugmentorExample");
      // Queries the Redis store for the top-N most similar chunks
      // to the user's query.
      EmbeddingStoreContentRetriever contentRetriever =
EmbeddingStoreContentRetriever.builder()
            .embeddingModel(model)
            .embeddingStore(store)
            .maxResults(3)
            .build();
      // Wraps the retriever and automatically prepends the
      // retrieved passages to your prompt before calling the LLM.
      augmentor = DefaultRetrievalAugmentor
            .builder()
            .contentRetriever(contentRetriever)
            .build();
      LOGGER.info("Retrieval AugmentorExample created");
   }
   @Override
   public RetrievalAugmentor get() {
      return augmentor;
   }
}
```

Enable RAG in Your AI Service

You have now set up the code to ingest documents when your application starts and set up the RetrievalAugmentor class that enables the RAG pattern in LangChain4j. The final missing piece is to enable RAG by passing in your RetrievalAugmentorExample class to the retrievalAugmentor parameter of the @RegisterAiService annotation. Also, to give insight to the LLM to support using the RAG searched results, you enhance your system and user prompts as shown next to "provide additional information as requested by the customer":

File: src/main/java/io/demo/langchain4j/sample/service/SupportService.java

```
package io.demo.langchain4j.sample.service;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import io.demo.langchain4j.sample.dto.CustomerRepository;
import io.demo.langchain4j.sample.rag.RetrievalAugmentorExample;
import io.quarkiverse.langchain4j.RegisterAiService;
@RegisterAiService(
      tools = {CustomerRepository.class},
      // Tells Quarkus to hook in our custom RetrievalAugmentor, which
      // will fetch relevant document snippets before each LLM call.
      retrievalAugmentor = RetrievalAugmentorExample.class
)
public interface SupportService {
   @SystemMessage("""
      You are working for an ecommerce company, processing comments
      about their products and providing additional information as
      requested by the customer. Categorize comments into positive,
      neutral and negative ones, responding with a JSON format
      document that includes a personalized greeting if you have the
      customer's details.
   """)
   @UserMessage("""
         Your task is to process the comment delimited by ---.
         Apply sentiment analysis to the comment to determine
         if it is positive, neutral or negative.
         For example:
         - `I love the user experience in your app, you are my best place to be!` is a
'POSITIVE'
         comment
         - `J'adore ton application` is a 'POSITIVE' comment
         - `I don't want to use your app, it's too slow!` is a 'NEGATIVE'
         comment
         Respond with a JSON format containing:
         - the 'rating' key set to 'POSITIVE' if the comment is
         positive, 'NEUTRAL' if the comment is neutral, 'NEGATIVE' otherwise
         - the 'message' key set to a message thanking or
         apologizing to the customer. These messages must be polite
         . Find the customer name if
         there is an id provided and use it to greet the customer.
         Also provide any additional information as requested by
         the customer.
         ---
         {comment}
         {customerId}
         ---
   """)
```

```
    CommentReview categorize(String comment, long customerId);
}
```

Let's boot up the application and comment the system logs. The output is a bit truncated due to the verbose display of the response from the OpenAI Embedding model. It shows that your file was read, and then the embeddings of it were requested from OpenAI Embedding API. Finally, the embedding documents are inserted into Redis.

Output:

```
        -0.061810866,
        0.019966396,
        0.0056421002,
        -0.02856894,
        -0.016740443,
        -0.0003667365,
        -0.0022900288,
        -0.025276609,
        0.025077475,
        0.0011923085,
        0.020470867,
        0.010666888,
        0.026657263,
        -0.016222697,
        0.0039162813,
        -0.026936049,
        0.0028874276,
        -0.0039793397,
        -0.012459084,
        -0.017709557
      ]
    }
  ],
  "model": "text-embedding-ada-002-v2",
  "usage": {
    "prompt_tokens": 145,
    "total_tokens": 145
  }
}
2025-04-23 18:28:15,726 INFO  [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus Main Thread) Documents ingested into Redis.
```

Now that your application has booted up and your documents have been ingested into Redis, let's run a curl POST request to see the output. This time, in addition to the

feedback comment, you will also ask to give the list of all the products in your e-commerce catalog.

```
curl localhost:8080/comment --header 'Content-Type: text/plain' --data 'This is a great product! I highly recommend it. Could you list all products from your shop catalog?'
```

Output:


```
{"rating":"POSITIVE","message":"Thank you for your wonderful feedback, John Doe! We're glad to hear that you highly recommend our product. Here is the list of products from our shop catalog: \n1. T-Shirt - Comfortable cotton t-shirt - $15.99\n2. Jeans - Classic blue denim jeans - $39.99\n3. Running Shoes - Lightweight running shoes - $79.99\n4. Laptop - 15-inch laptop with SSD - $999.00\n5. Coffee Maker - Automatic coffee maker - $49.99"}
```


As you can see from the output, not only is there personalized feedback using the customer name and rating, but also there is a complete list of products from your company's catalog.

Let's also see what is happening in the system backend while you are seeing this response. As you can see from the truncated numbers, the user query is first converted into embeddings by making a call to the OpenAI Embedding Model API and receives back the query in embeddings format.

Output:


```
      -0.044992004,
      -0.005532781,
      0.0048767063,
      0.004967925,
      0.017991187,
      -0.034382533,
      -0.0011218179,
      0.0018033289,
      0.030004025,
      0.0024892252,
      0.019548927,
      -0.04041702
    ]
   }
 ],
 "model": "text-embedding-ada-002-v2",
```

```
  "usage": {
    "prompt_tokens": 272,
    "total_tokens": 272
  }
}
```

The embeddings are matched with the Redis store, and the returned documents are passed in your prompt to the OpenAI API.

Output:

```
2025-04-23 18:32:20,862
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-thread-3) Request:
- method: POST
- url: https://api.openai.com/v1/chat/completions
- headers: [Accept: application/json], [Authorization: Be...kA], [Content-Type: application/json], [User-Agent: langchain4j-openai], [content-length: 2539]
- body: {
  "model" : "gpt-4o-mini",
  "messages" : [ {
    "role" : "system",
    "content" : "       You are working for an ecommerce company, processing comments about\n       their products and providing additional information as requested\n       \tby the customer. Categorize comments into positive, neutral and negative ones,\nresponding with a JSON format document.\n"
  }, {
    "role" : "user",
    "content" : "       Your task is to process the comment delimited by ---.\n       Apply sentiment analysis to the comment to determine\n       if it is positive, neutral or negative.\n\n       For example:\n       - `I love the user experience in your app, you are my best place to be!` is a 'POSITIVE' comment\n       - `J'adore ton application` is a 'POSITIVE' comment\n       - `I don't want to use your app, it's too slow!` is a 'NEGATIVE' comment\n\n       Respond with a JSON format containing:\n       - the 'rating' key set to 'POSITIVE' if the comment is\n       positive, 'NEUTRAL' if the comment is neutral, 'NEGATIVE' otherwise\n       - the 'message' key set to a message thanking or apologizing\n       to the customer. These messages must be polite. Find the customer name if there is an id provided\n       and use it to greet the customer. Also provide any additional\n information as requested by the customer.\n\n       ---\n       This is a great product! I highly recommend it. Could you list all products from your shop catalog?\n       1\n       ---\n\nYou must answer strictly in the following JSON format: {\n\"rating\": (type: enum, must be one of [POSITIVE, NEUTRAL, NEGATIVE]),\n\"message\": (type: string)\n}\n\nAnswer using the following information:\nProductID,Name,Description,Price,Category,Manufacturer,SKU\n1001,T-Shirt,Comfortable cotton t-shirt,15.99,Clothing,BrandA,TS-
```

```
1001\n1002,Jeans,Classic blue denim jeans,39.99,Clothing,BrandB,JN-
1002\n1003,Running Shoes,Lightweight running shoes,79.99,Shoes,BrandC,RS-
1003\n1004,Laptop,15-inch laptop with SSD,999.00,Electronics,BrandD,LP-
1004\n1005,Coffee Maker,Automatic coffee maker,49.99,Appliances,BrandE,CM-
1005"
 } ],
 "temperature" : 0.5,
 "top_p" : 1.0,
 "presence_penalty" : 0.0,
 "frequency_penalty" : 0.0,
 "tools" : [ {
  "type" : "function",
  "function" : {
   "name" : "getCustomerDetail",
   "description" : "get the customer detail for the given customerId",
   "parameters" : {
    "type" : "object",
    "properties" : {
     "id" : {
      "type" : "integer"
     }
    },
    "required" : [ "id" ]
   }
  }
 } ]
}
```


OpenAI uses the tool call to fetch the customer name information.

Output:


```
2025-04-23 18:32:22,233
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-
thread-3) Response:
- status code: 200
- headers: [Date: Wed, 23 Apr 2025 18:32:22 GMT], [Content-Type: application/json],
[Transfer-Encoding: chunked], [Connection: keep-alive], [access-control-expose-
headers: X-Request-ID], [openai-organization: user-eldkOkvytvetb2vdgo2y3nvr],
[openai-processing-ms: 758], [openai-version: 2020-10-01], [x-ratelimit-limit-
requests: 10000], [x-ratelimit-limit-tokens: 200000], [x-ratelimit-remaining-
requests: 9999], [x-ratelimit-remaining-tokens: 199523], [x-ratelimit-reset-requests:
8.64s], [x-ratelimit-reset-tokens: 142ms], [x-request-id:
req_43a545a265819b93d4ca7379be2e7344], [strict-transport-security: max-
age=31536000; includeSubDomains; preload], [cf-cache-status: DYNAMIC], [Set-
```

Cookie: __...ne], [X-Content-Type-Options: nosniff], [Set-Cookie: _c...ne], [Server: cloudflare], [CF-RAY: 934f716a6974d352-FRA], [alt-svc: h3=":443"; ma=86400]
- body: {
 "id": "chatcmpl-BPZBNOtOVxWBJMEMAgaHZIOTmH9Xt",
 "object": "chat.completion",
 "created": 1745433141,
 "model": "gpt-4o-mini-2024-07-18",
 "choices": [
  {
   "index": 0,
   "message": {
    "role": "assistant",
    "content": null,
    "tool_calls": [
     {
      "id": "call_0QGLPggBPNP26aUqRV4rwwQD",
      "type": "function",
      "function": {
       "name": "getCustomerDetail",
       "arguments": "{\"id\": 1}"
      }
     }
    ],
    "refusal": null,
    "annotations": []
   },
   "logprobs": null,
   "finish_reason": "tool_calls"
  }
 ],
 "usage": {
  "prompt_tokens": 512,
  "completion_tokens": 32,
  "total_tokens": 544,
  "prompt_tokens_details": {
   "cached_tokens": 0,
   "audio_tokens": 0
  },
  "completion_tokens_details": {
   "reasoning_tokens": 0,
   "audio_tokens": 0,
   "accepted_prediction_tokens": 0,
   "rejected_prediction_tokens": 0
  }
 },
 "service_tier": "default",

```
  "system_fingerprint": "fp_dbaca60df0"
}
```

A final call is made to OpenAI using the response from the database fetch query tool and the embeddings matched documents.

Output:

```
2025-04-23 18:32:22,884
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-thread-3) Request:
- method: POST
- url: https://api.openai.com/v1/chat/completions
- headers: [Accept: application/json], [Authorization: Be...kA], [Content-Type: application/json], [User-Agent: langchain4j-openai], [content-length: 2890]
- body: {
  "model" : "gpt-4o-mini",
  "messages" : [ {
    "role" : "system",
    "content" : "       You are working for an ecommerce company, processing comments about\n       their products and providing additional information as requested\n       \tby the customer. Categorize comments into positive, neutral and negative ones,\nresponding with a JSON format document.\n"
  }, {
    "role" : "user",
    "content" : "       Your task is to process the comment delimited by ---.\n       Apply sentiment analysis to the comment to determine\n       if it is positive, neutral or negative.\n\n       For example:\n       - `I love the user experience in your app, you are my best place to be!` is a 'POSITIVE' comment\n       - `J'adore ton application` is a 'POSITIVE' comment\n       - `I don't want to use your app, it's too slow!` is a 'NEGATIVE' comment\n\n       Respond with a JSON format containing:\n       - the 'rating' key set to 'POSITIVE' if the comment is\n       positive, 'NEUTRAL' if the comment is neutral, 'NEGATIVE' otherwise\n       - the 'message' key set to a message thanking or apologizing\n       to the customer. These messages must be polite. Find the customer name if there is an id provided\n       and use it to greet the customer. Also provide any additional\n information as requested by the customer.\n\n       ---\n       This is a great product! I highly recommend it. Could you list all products from your shop catalog?\n       1\n       ---\n\nYou must answer strictly in the following JSON format: {\n\"rating\": (type: enum, must be one of [POSITIVE, NEUTRAL, NEGATIVE]),\n\"message\": (type: string)\n}\n\nAnswer using the following information:\nProductID,Name,Description,Price,Category,Manufacturer,SKU\n1001,T-Shirt,Comfortable cotton t-shirt,15.99,Clothing,BrandA,TS-1001\n1002,Jeans,Classic blue denim jeans,39.99,Clothing,BrandB,JN-1002\n1003,Running Shoes,Lightweight running shoes,79.99,Shoes,BrandC,RS-1003\n1004,Laptop,15-inch laptop with SSD,999.00,Electronics,BrandD,LP-
```

```
1004\n1005,Coffee Maker,Automatic coffee maker,49.99,Appliances,BrandE,CM-
1005"
 }, {
  "role" : "assistant",
  "tool_calls" : [ {
   "id" : "call_OQGLPggBPNP26aUqRV4rwwQD",
   "type" : "function",
   "function" : {
    "name" : "getCustomerDetail",
    "arguments" : "{\"id\": 1}"
   }
  } ]
 }, {
  "role" : "tool",
  "tool_call_id" : "call_OQGLPggBPNP26aUqRV4rwwQD",
  "content" : "\"John Doe\""
 } ],
 "temperature" : 0.5,
 "top_p" : 1.0,
 "presence_penalty" : 0.0,
 "frequency_penalty" : 0.0,
 "tools" : [ {
  "type" : "function",
  "function" : {
   "name" : "getCustomerDetail",
   "description" : "get the customer detail for the given customerId",
   "parameters" : {
    "type" : "object",
    "properties" : {
     "id" : {
      "type" : "integer"
     }
    },
    "required" : [ "id" ]
   }
  }
 } ]
}
```

Here you receive the final augmented response from the OpenAI LLM:

Output:

```
2025-04-23 18:32:26,452
INFO  [io.qua.lan.ope.com.OpenAiRestApi$OpenAiClientLogger] (vert.x-eventloop-
```

thread-3) Response:
- status code: 200
- headers: [Date: Wed, 23 Apr 2025 18:32:26 GMT], [Content-Type: application/json], [Transfer-Encoding: chunked], [Connection: keep-alive], [access-control-expose-headers: X-Request-ID], [openai-organization: user-eldk0kvytvetb2vdgo2y3nvr], [openai-processing-ms: 3429], [openai-version: 2020-10-01], [x-ratelimit-limit-requests: 10000], [x-ratelimit-limit-tokens: 200000], [x-ratelimit-remaining-requests: 9998], [x-ratelimit-remaining-tokens: 199520], [x-ratelimit-reset-requests: 15.645s], [x-ratelimit-reset-tokens: 144ms], [x-request-id: req_936c2de81360e8e28f1ac84788c49243], [strict-transport-security: max-age=31536000; includeSubDomains; preload], [cf-cache-status: DYNAMIC], [Set-Cookie: __...ne], [X-Content-Type-Options: nosniff], [Set-Cookie: _c...ne], [Server: cloudflare], [CF-RAY: 934f71771a45d352-FRA], [alt-svc: h3=":443"; ma=86400]
- body: {
  "id": "chatcmpl-BPZBPFEgZYBwIf6DrpBDPNvZuWYBI",
  "object": "chat.completion",
  "created": 1745433143,
  "model": "gpt-4o-mini-2024-07-18",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "**{\n \"rating\": \"POSITIVE\",\n \"message\": \"Thank you for your wonderful feedback, John Doe! We're glad to hear that you highly recommend our product. Here is the list of products from our shop catalog: \\n1. T-Shirt - Comfortable cotton t-shirt - $15.99\\n2. Jeans - Classic blue denim jeans - $39.99\\n3. Running Shoes - Lightweight running shoes - $79.99\\n4. Laptop - 15-inch laptop with SSD - $999.00\\n5. Coffee Maker - Automatic coffee maker - $49.99\"\n}**",
        "refusal": null,
        "annotations": []
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 540,
    "completion_tokens": 124,
    "total_tokens": 664,
    "prompt_tokens_details": {
      "cached_tokens": 0,
      "audio_tokens": 0
    },
    "completion_tokens_details": {

```
      "reasoning_tokens": 0,
      "audio_tokens": 0,
      "accepted_prediction_tokens": 0,
      "rejected_prediction_tokens": 0
    }
  },
  "service_tier": "default",
  "system_fingerprint": "fp_dbaca60df0"
}
```

So, what happens at runtime?

1. **Service invocation**: categorize(comment, customerId) is called.
2. **Retrieval**: RetrievalAugmentorExample fetches up to three catalog passages most semantically similar to comment.
3. **Prompt assembly**: The system message, any chat memory, the retrieved snippets, and the user prompt (with {comment} and {customerId}) are concatenated.
4. **LLM call**: The enriched prompt is sent to the model, which can now ground its analysis in your catalog data (e.g., product specs, descriptions).
5. **Response parsing**: The JSON output is deserialized into CommentReview.

By wiring in RAG, you ensure that your AI assistant leverages your fresh, proprietary content—whether it's product catalogs, FAQs, or knowledge bases—yielding more accurate, context-aware responses without manual prompt concatenation or bespoke retrieval code.

Conclusion

In this chapter, you walked through every stage of embedding LLMs into a Quarkus application—from the initial LangChain4j setup and precise prompt engineering to the dynamic tool integration, conversational memory, and advanced RAG techniques. You now know how to register AI services with just a few annotations, enrich prompts with real-time data via custom tools, and ground your model's outputs in external knowledge bases. The sample application illustrated not only sentiment categorization but also how to persist context and perform semantic retrieval to create richer, more accurate responses.

Armed with these patterns and code samples, you're ready to build your own LLM-infused microservices on Quarkus. Whether you're processing customer feedback, powering chatbots, or generating personalized recommendations, the combination of

Quarkus's optimized runtime and LangChain4j's Java-native abstractions gives you a solid foundation for scalable, maintainable AI systems.

In the next chapter, you'll explore hosting models in Java. Keep experimenting, tuning, and extending—your journey into cloud-native Gen AI is just beginning!

Previous chapter
< 11. Agents Assemble! Building Autonomous Workflows

Next chapter
13. Jlama and Friends: Hosting Models the Java Way >